

シミュレーション理工学 レポートその 1

特別聴講学生（東京学芸大学）

情報理工学域

26A2001 樋田巧

提出日 2026 年 6 月 12 日

1 演習 1 問題 2

1.1 課題

オイラーリチャードソン法を用い、惑星の回転運動をシミュレーションするプログラムを作成する。作成したプログラムを用いて、時間刻みによる最大の回転半径の変化を調べよ（時間刻みは 2 桁程度変化させて調べること）。

1.2 計算原理と用いた式

太陽を原点に固定し、惑星の運動のみを考える。万有引力の法則とニュートンの運動方程式より、惑星の運動は

$$\frac{d^2x}{dt^2} = -\frac{GM}{r^3}x \quad (1)$$

$$\frac{d^2y}{dt^2} = -\frac{GM}{r^3}y \quad (2)$$

で表される。ただし、

$$r = \sqrt{x^2 + y^2} \quad (3)$$

である。

今回の授業では、長さを 1 AU，時間を 1 年で規格化している。このとき、

$$GM = 4\pi^2 \quad (4)$$

となるため、プログラムではこの値を用いた。

また、2 階微分方程式を 1 階連立微分方程式に変換すると、

$$\frac{dx}{dt} = v_x \quad (5)$$

$$\frac{dy}{dt} = v_y \quad (6)$$

$$\frac{dv_x}{dt} = -\frac{GM}{r^3}x \quad (7)$$

$$\frac{dv_y}{dt} = -\frac{GM}{r^3}y \quad (8)$$

となる。これらの式をオイラー・リチャードソン法によって数値的に解いた。

1.3 プログラムの考え方

初期条件として,

$$x = 1, \quad y = 0$$

$$v_x = 0, \quad v_y = 2\pi$$

を与えた。

まず現在位置から加速度を求め、その値を用いて時間刻みの半分だけ進めた中間点の位置と速度を計算する。その後、中間点での加速度を求め、その値を用いて位置と速度を更新した。

時間刻み Δt は 0.001 から 0.100 まで変化させ、各条件についてシミュレーションを行った。計算中に半径

$$r = \sqrt{x^2 + y^2}$$

を毎ステップ計算し、その最大値を $r_{\max}$ として記録した。

得られた Δt と $r_{\max}$ の関係を出力し、gnuplot を用いてグラフ化した。

1.4 実行結果と考察

図 1 に時間刻み Δt と最大回転半径 $r_{\max}$ の関係を示す。

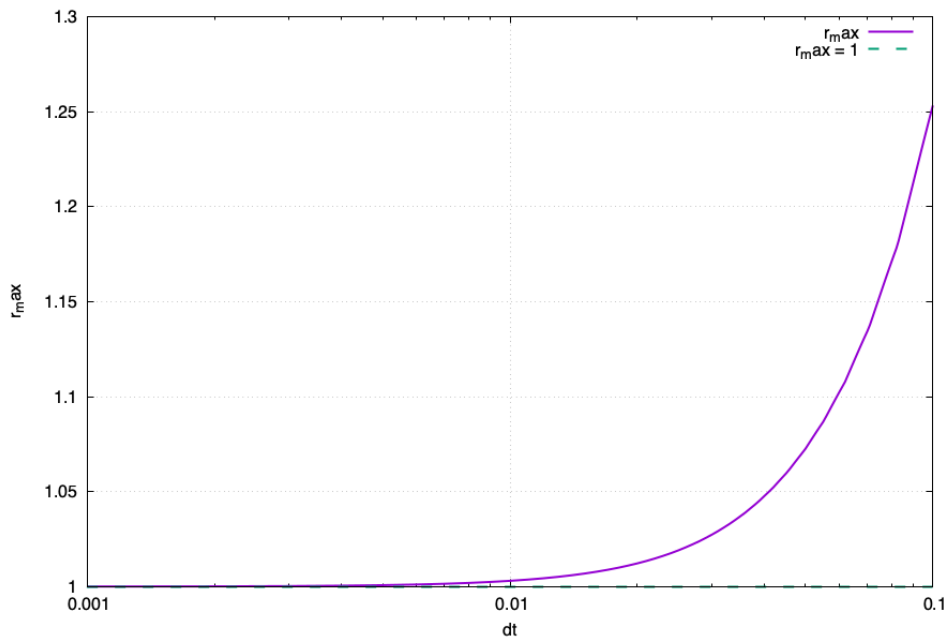


図1 時間刻み Δt と最大回転半径 $r_{\max}$ の関係

図より、時間刻みが大きい場合は $r_{\max}$ が 1 から大きくずれていることが分かる。一方で、時間刻みを小さくするにつれて $r_{\max}$ は 1 に近づいている。

理想的な円軌道では半径は常に 1 AU であるため、最大回転半径も 1 となるはずである。しかし、数値計算では時間を離散的に扱うため、時間刻みが大きいと近似誤差が大きくなり、軌道が理想的な円軌道からずれてしまう。その結果、最大回転半径も大きくなったと考えられる。

今回の結果から、オイラー・リチャードソン法では時間刻みを小さくすることで計算精度が向上し、理論値に近い結果が得られることを確認できた。

2 演習 2 問題 3

2.1 課題

講義資料「2 重星」の問題に対するシミュレーションを行い、「軌道の分類」に対応する結果を求めよ。注：この結果は惑星軌道の判定法に依存するので、必ずしも全く同じ結果が得られるとは限らない。

2.2 計算原理と用いた式

座標平面上に 2 つの恒星を固定し、その位置を

$$(-1, 0), (1, 0)$$

とした。

質点は 2 つの恒星から重力を受けるため、それぞれの恒星との距離を

$$r_1 = \sqrt{(x+1)^2 + y^2}$$

$$r_2 = \sqrt{(x-1)^2 + y^2}$$

とすると、加速度は

$$\frac{dv_x}{dt} = -\frac{x+1}{r_1^3} - \frac{x-1}{r_2^3} \quad (9)$$

$$\frac{dv_y}{dt} = -\frac{y}{r_1^3} - \frac{y}{r_2^3} \quad (10)$$

で表される。

また、位置と速度の関係は

$$\frac{dx}{dt} = v_x \quad (11)$$

$$\frac{dy}{dt} = v_y \quad (12)$$

である。

これらの連立1階微分方程式をオイラー・リチャードソン法によって数値的に解いた。

2.3 プログラムの考え方

質点の初期位置は

$$x = 0.1, \quad y = 1.0$$

とした。

また、初速度の向きや大きさを変化させて計算を行った。

各時刻において現在位置から加速度を求め、その値を用いて時間刻みの半分だけ進めた中間点の位置と速度を計算した。その後、中間点での加速度を求め、その値を用いて位置と速度を更新した。

時間刻みは

$$\Delta t = 0.001$$

とし、

$$t = 10$$

まで計算を行った。

初速度の向きや大きさを複数変更して計算を行い、その中から軌道の特徴が分かりやすい結果を選択した。

2.4 実行結果と考察

本シミュレーションでは、初速度の向きとして水平方向、鉛直方向、斜め方向の3種類を設定し、さらにそれぞれについて速度成分の大きさを12通り変化させた。したがって、合計36通りの初期条件について計算を行った。

その結果、多くの条件では質点が二重星系から遠ざかり、閉じた軌道を形成せずに発散する様子が見られた。一方で、いくつかの条件では特徴的な軌道が確認できたため、その代表例を図2～図4に示す。

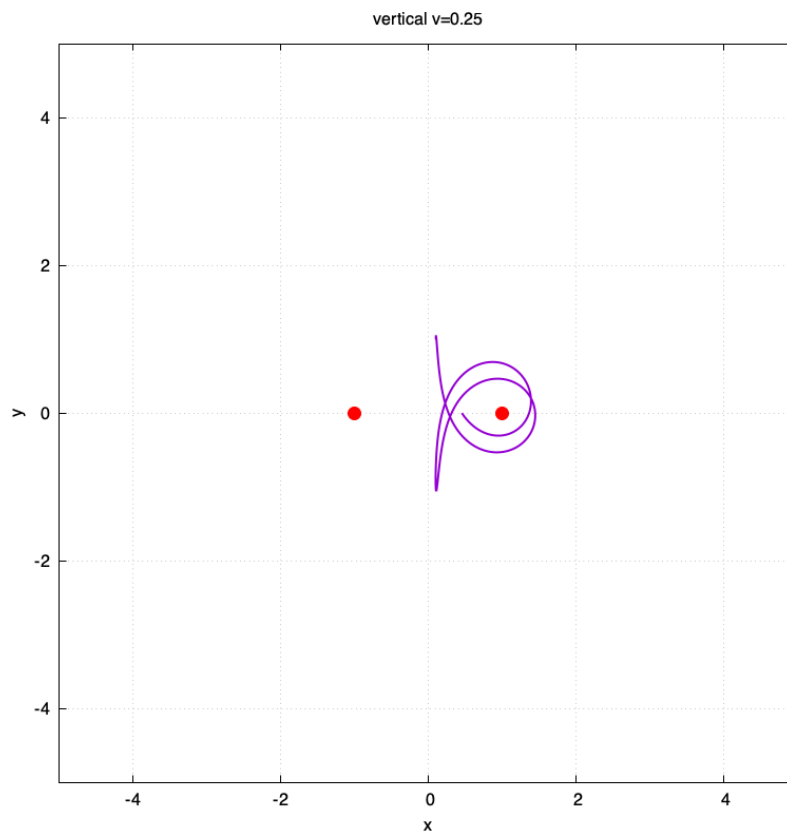


図2 鉛直方向の初速度を与えた場合

図2では、質点は右側の恒星の周囲を回るような軌道を描いていることが分かる。

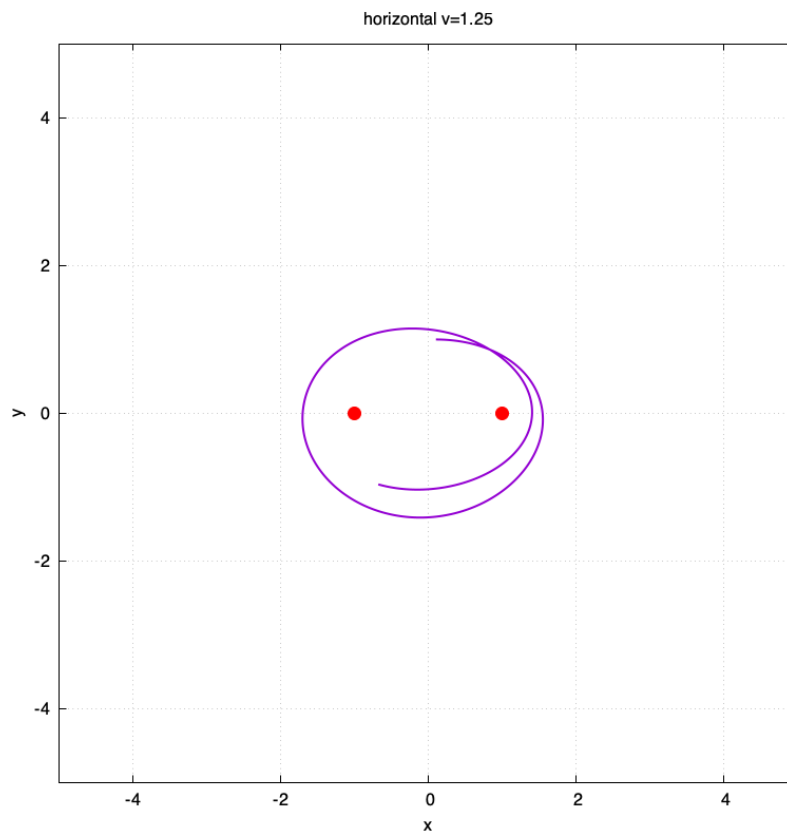


図 3 水平方向の初速度を与えた場合

図 3 では、質点は 2 つの恒星をまとめて囲むような軌道を描いている。

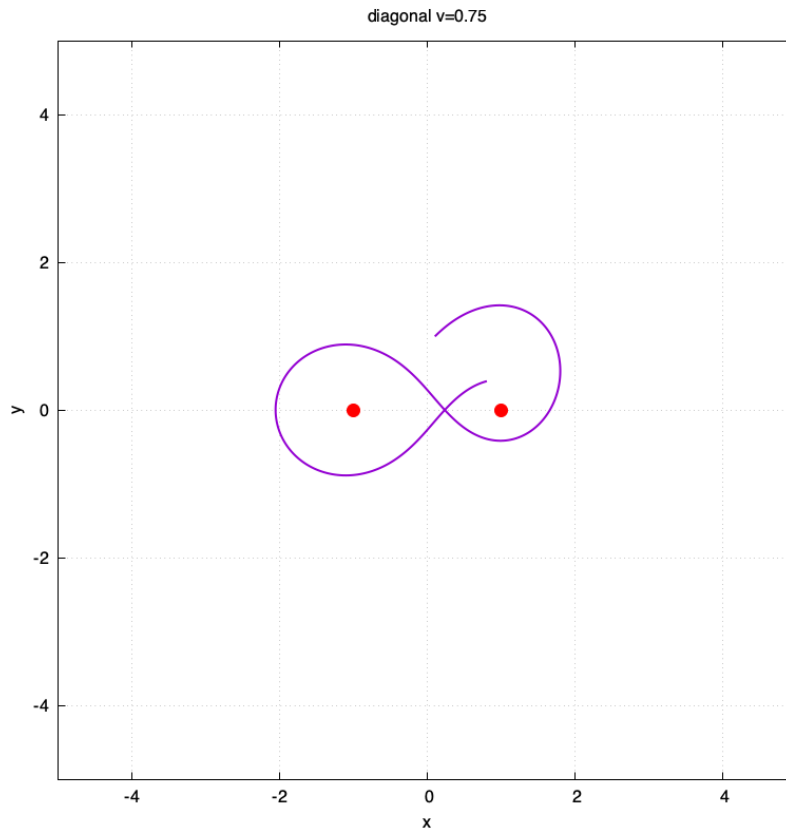


図 4 斜め方向の初速度を与えた場合

図 4 では、質点が 2 つの恒星の間を通過しながら運動し、8 の字のような軌道を描いていることが確認できる。

今回のシミュレーションでは、初速度の向きや速度成分の設定を変えることで軌道の形が大きく変化した。多くの場合は質点が系外へ飛び出してしまったが、一部の条件では恒星の重力によって束縛され、特徴的な軌道を形成した。

鉛直方向に初速度を与えた場合は一方の恒星の周囲を回る軌道となり、水平方向に初速度を与えた場合は 2 つの恒星を囲む軌道となった。また、斜め方向に初速度を与えた場合には、両方の恒星の近くを通過する 8 の字状の軌道が確認できた。

このことから、二重星系における質点運動では初期条件によって軌道の性質が大きく変化することが分かった。また、同じ重力場であっても、一方の恒星を周回する軌道、二重星全体を周回する軌道、両恒星の間を通過する軌道など、多様な運動が現れることを確認できた。

3 演習 3 問題 3

3.1 課題

適応的時間幅アルゴリズムを用いて、講義資料「三体問題」の問題の中から問題を 2 つ選択してシミュレーションを行い、適応的時間幅アルゴリズムの効果を調査せよ。

3.2 計算原理と用いた式

電子 1, 電子 2 の位置をそれぞれ

$$\mathbf{r}_1 = (x_1, y_1)$$

$$\mathbf{r}_2 = (x_2, y_2)$$

とする。

このとき, 電子 1, 電子 2 の運動方程式は

$$\frac{d^2 \mathbf{r}_1}{dt^2} = -2 \frac{\mathbf{r}_1}{|\mathbf{r}_1|^3} + \frac{\mathbf{r}_1 - \mathbf{r}_2}{|\mathbf{r}_1 - \mathbf{r}_2|^3} \quad (13)$$

$$\frac{d^2 \mathbf{r}_2}{dt^2} = -2 \frac{\mathbf{r}_2}{|\mathbf{r}_2|^3} - \frac{\mathbf{r}_1 - \mathbf{r}_2}{|\mathbf{r}_1 - \mathbf{r}_2|^3} \quad (14)$$

で表される。

本課題では, これらの式をオイラー・リチャードソン法によって数値的に解いた。

3.3 プログラムの考え方

初期条件として, 初期条件 A では

$$\mathbf{r}_1 = (2.0, 0), \quad \mathbf{r}_2 = (-1.0, 0)$$

$$\mathbf{v}_1 = (0, 0.95), \quad \mathbf{v}_2 = (0, -1)$$

を与えた。

また, 初期条件 B では

$$\mathbf{r}_1 = (1.4, 0), \quad \mathbf{r}_2 = (-1.0, 0)$$

$$\mathbf{v}_1 = (0, 0.86), \quad \mathbf{v}_2 = (0, -1)$$

を与えた。

運動方程式は演習 1 と同様にオイラー・リチャードソン法によって数値的に解いた。

固定時間幅法では,

$$\Delta t = 0.001$$

を一定として計算を行った。

一方, 適応時間幅法では, 同じ時間区間を

$$dt$$

で 1 回計算した結果と、

$$\frac{dt}{2}$$

で 2 回計算した結果を比較し、その差を局所誤差の指標とした。

誤差が設定値より大きい場合には時間幅を半分にし、十分小さい場合には時間幅を 2 倍にすることで、計算中に時間幅を自動的に調整した。

計算時間は

$$t = 80$$

までとし、各時刻における電子の位置、エネルギー、および初期エネルギーとの差である `energy_error` を出力した。

その後、固定時間幅法と適応時間幅法について、電子軌道および `energy_error` の比較を行った。

3.4 実行結果と考察

図 5 に初期条件 A における軌道を示す。

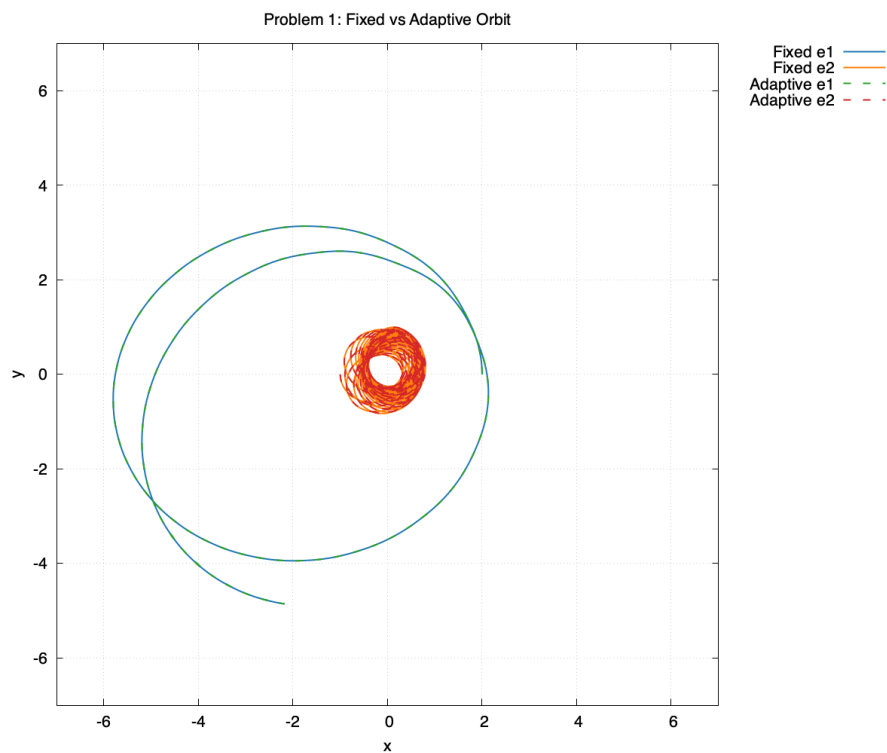


図5 初期条件 A における軌道の比較

初期条件 A では、固定時間幅と適応時間幅で軌道の始点および電子の運動の大まかな特徴は共通しているものの、長時間計算では軌道は完全には一致していないことが分かる。特に電子 2 の軌道では、固定時間幅と適応時間幅で軌道半径や軌跡に違いが見られた。一方で、どちらの場合も電子 1 は大きな軌道を描き、電子 2 は原点付近を周回していることから、運動の大まかな特徴は保たれていると考えられる。

図 6 に初期条件 B における軌道を示す。

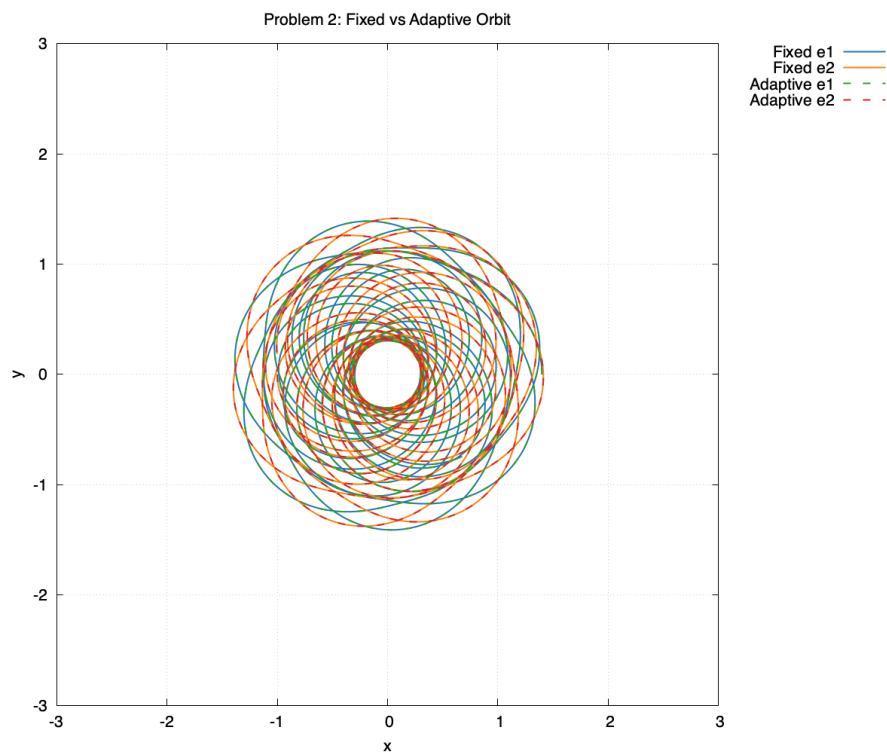


図6 初期条件 B における軌道の比較

初期条件 B では、固定時間幅と適応時間幅の軌道はほぼ重なっており、大きな違いは見られなかった。したがって、この条件では時間幅の制御方法による軌道への影響は比較的小さいと考えられる。

次に、初期エネルギーとの差である `energy_error` を比較した。

図 7 に初期条件 A における `energy_error` を示す。

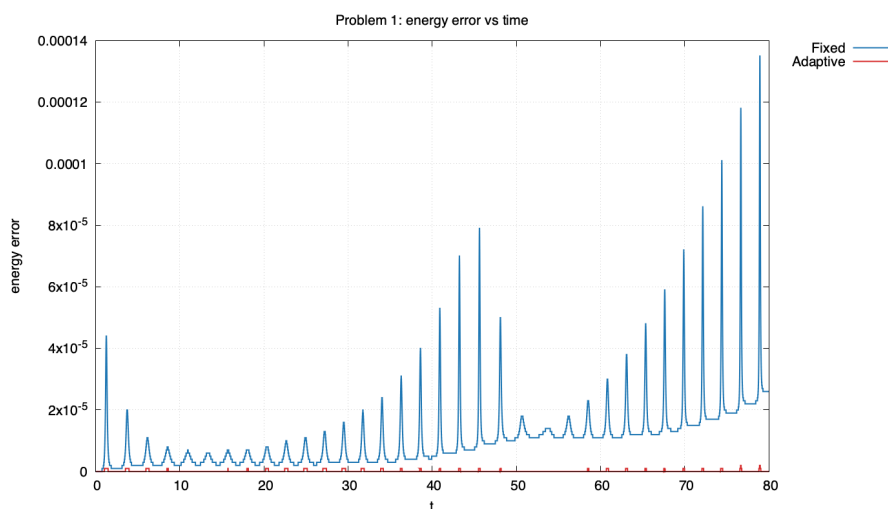


図7 初期条件 A における energy_error の比較

固定時間幅では energy_error が時間とともに増加しており、周期的に大きなピークが現れていることが分かる。一方、適応時間幅では energy_error が全体を通して非常に小さく抑えられている。

初期条件 A では、固定時間幅の最大 energy_error は

$$1.35 \times 10^{-4}$$

であった。一方、適応時間幅の最大 energy_error は

$$2.0 \times 10^{-6}$$

であり、誤差が大幅に減少していることが確認できた。

図8に初期条件 B における energy_error を示す。

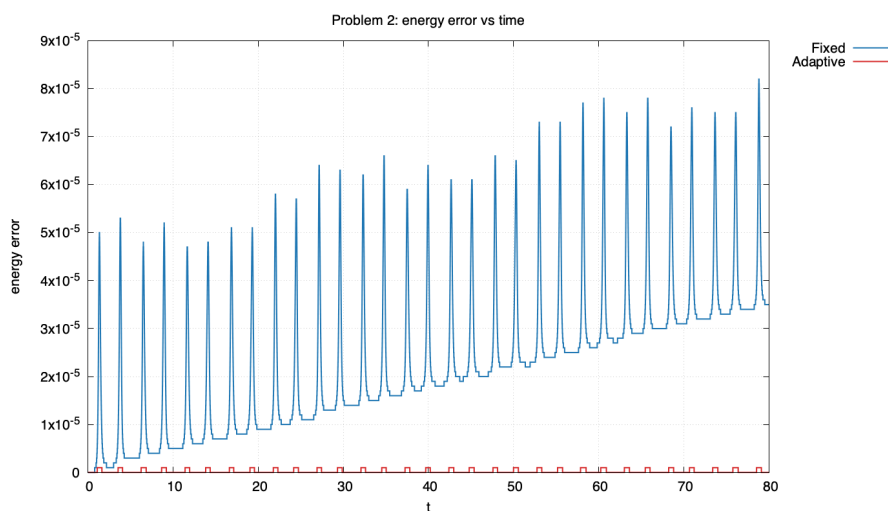


図8 初期条件 B における energy_error の比較

初期条件 B においても、固定時間幅では `energy_error` が徐々に増加している。一方、適応時間幅では `energy_error` が非常に小さく保たれている。

初期条件 B では、固定時間幅の最大 `energy_error` は

$$8.2 \times 10^{-5}$$

であった。一方、適応時間幅の最大 `energy_error` は

$$1.0 \times 10^{-6}$$

であり、こちらも固定時間幅より小さい値となった。

以上の結果より、初期条件 A、初期条件 B のどちらについても、今回用いた固定時間幅 $dt = 0.001$ と比較して、適応時間幅を用いた場合の方が `energy_error` を大幅に減少させることができた。特に初期条件 A では軌道そのものにも違いが見られたことから、長時間計算では誤差の蓄積が軌道へ影響を与えることが分かる。

一方、適応時間幅では運動の変化が大きい部分で時間幅を小さくし、変化が小さい部分で時間幅を大きくすることで、計算精度を保ちながら誤差の増大を抑えることができた。

このことから、適応時間幅アルゴリズムは今回の電子軌道計算条件において有効であり、固定時間幅 $dt = 0.001$ の場合よりも高い精度でシミュレーションを行えたことが確認できた。

4 プログラム

4.1 演習 1 問題 2

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 #define GM (4.0 * M_PI * M_PI)
6
7 double eulerRichardson(const double dt);
8
9 int main(void) {
10     FILE *radius_fp = fopen("r_max_data.txt", "w");
11
12     if (radius_fp == NULL) {
13         return 1;
14     }
15
16     for (int i = 100; i >= 1; i--) {
17         const double dt = i * 0.001;
18         const double r_max = eulerRichardson(dt);
19         fprintf(radius_fp, "%f□%f\n", dt, r_max);
20     }
21
22     fclose(radius_fp);
23     return 0;
```

```

24 }
25
26 double eulerRichardson(const double dt) {
27     const double t_max = 1.0;
28     const int max = (int)(t_max / dt);
29
30     double x = 1.0;
31     double y = 0.0;
32     double vx = 0.0;
33     double vy = 2.0 * M_PI;
34     double r_max = 0.0;
35
36     for (int i = 0; i < max; i++) {
37         const double r = sqrt(x * x + y * y);
38
39         if (r > r_max) {
40             r_max = r;
41         }
42
43         const double r3 = r * r * r;
44         const double ax = -GM * x / r3;
45         const double ay = -GM * y / r3;
46
47         const double x_mid = x + 0.5 * vx * dt;
48         const double y_mid = y + 0.5 * vy * dt;
49         const double vx_mid = vx + 0.5 * ax * dt;
50         const double vy_mid = vy + 0.5 * ay * dt;
51
52         const double r_mid = sqrt(x_mid * x_mid + y_mid * y_mid);
53         const double r3_mid = r_mid * r_mid * r_mid;
54         const double ax_mid = -GM * x_mid / r3_mid;
55         const double ay_mid = -GM * y_mid / r3_mid;
56
57         vx += ax_mid * dt;
58         vy += ay_mid * dt;
59         x += vx_mid * dt;
60         y += vy_mid * dt;
61     }
62
63     return r_max;
64 }

```

4.2 演習 2 問題 3

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>

```

```

4
5 #define GM (1.0)
6 #define STAR_DISTANCE (1.0)
7
8 enum State { X, Y, N };
9
10 void calcAcceleration(const double pos[N], double acc[N]);
11 void eulerRichardson(const char filename[], const double vx0, const double vy0)
12     ;
13
14 int main(void) {
15     const double v_list[] = {0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0,
16         2.25, 2.5, 2.75, 3.0};
17     const int v_num = 12;
18
19     for (int i = 0; i < v_num; i++) {
20         char filename[100];
21
22         sprintf(filename, "orbit_horizontal_%d.dat", i);
23         eulerRichardson(filename, v_list[i], 0.0);
24
25         sprintf(filename, "orbit_vertical_%d.dat", i);
26         eulerRichardson(filename, 0.0, v_list[i]);
27
28         sprintf(filename, "orbit_diagonal_%d.dat", i);
29         eulerRichardson(filename, v_list[i], v_list[i]);
30     }
31
32     return 0;
33 }
34
35 void calcAcceleration(const double pos[N], double acc[N]) {
36     const double star1[N] = {-STAR_DISTANCE, 0.0};
37     const double star2[N] = { STAR_DISTANCE, 0.0};
38
39     acc[X] = 0.0;
40     acc[Y] = 0.0;
41
42     const double dx1 = star1[X] - pos[X];
43     const double dy1 = star1[Y] - pos[Y];
44     const double r1 = sqrt(dx1 * dx1 + dy1 * dy1);
45     const double r13 = r1 * r1 * r1;
46
47     acc[X] += GM * dx1 / r13;
48     acc[Y] += GM * dy1 / r13;
49
50     const double dx2 = star2[X] - pos[X];
51     const double dy2 = star2[Y] - pos[Y];
52     const double r2 = sqrt(dx2 * dx2 + dy2 * dy2);

```

```

51     const double r23 = r2 * r2 * r2;
52
53     acc[X] += GM * dx2 / r23;
54     acc[Y] += GM * dy2 / r23;
55 }
56
57 void eulerRichardson(const char filename[], const double vx0, const double vy0)
58 {
59     FILE *fp = fopen(filename, "w");
60
61     if (fp == NULL) {
62         return;
63     }
64
65     const double dt = 0.001;
66     const double t_max = 10.0;
67     const int max = (int)(t_max / dt);
68
69     double pos[N] = {0.1, 1.0};
70     double vel[N] = {vx0, vy0};
71     double acc[N] = {0.0, 0.0};
72
73     for (int i = 0; i < max; i++) {
74         calcAcceleration(pos, acc);
75
76         const double pos_mid[N] = {
77             pos[X] + 0.5 * vel[X] * dt,
78             pos[Y] + 0.5 * vel[Y] * dt
79         };
80
81         const double vel_mid[N] = {
82             vel[X] + 0.5 * acc[X] * dt,
83             vel[Y] + 0.5 * acc[Y] * dt
84         };
85
86         double acc_mid[N] = {0.0, 0.0};
87         calcAcceleration(pos_mid, acc_mid);
88
89         vel[X] += acc_mid[X] * dt;
90         vel[Y] += acc_mid[Y] * dt;
91
92         pos[X] += vel_mid[X] * dt;
93         pos[Y] += vel_mid[Y] * dt;
94
95         fprintf(fp, "%f%f\n", pos[X], pos[Y]);
96     }
97
98     fclose(fp);
99 }

```

4.3 演習 3 問題 3

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 #define Z (2.0)
6 #define DT (0.001)
7 #define DT_MIN (0.000001)
8 #define DT_MAX (0.004)
9 #define T_MAX (80.0)
10 #define ERROR_LIMIT (0.00000001)
11 #define NE (2)
12
13 #define PROBLEM1 (1)
14 #define PROBLEM2 (2)
15
16 enum State { X, Y, N };
17 enum Electron { E1, E2 };
18
19 void setInitialCondition(
20     const int problem,
21     double pos[NE][N],
22     double vel[NE][N]
23 );
24 void calcAcceleration(
25     const double pos[NE][N],
26     double acc[NE][N]
27 );
28 void eulerRichardsonStep(
29     const double pos[NE][N],
30     const double vel[NE][N],
31     const double dt,
32     double next_pos[NE][N],
33     double next_vel[NE][N]
34 );
35 double calcEnergy(
36     const double pos[NE][N],
37     const double vel[NE][N]
38 );
39 double calcDifference(
40     const double pos1[NE][N],
41     const double pos2[NE][N],
42     const double vel1[NE][N],
43     const double vel2[NE][N]
44 );
```

```

45 void copyState(
46     const double src_pos[NE][N],
47     const double src_vel[NE][N],
48     double dst_pos[NE][N],
49     double dst_vel[NE][N]
50 );
51 void fixedStepSimulation(
52     const char filename[],
53     const int problem
54 );
55 void adaptiveStepSimulation(
56     const char filename[],
57     const int problem
58 );
59
60 int main(void) {
61     fixedStepSimulation("problem1_fixed.dat", PROBLEM1);
62     adaptiveStepSimulation("problem1_adaptive.dat", PROBLEM1);
63
64     fixedStepSimulation("problem2_fixed.dat", PROBLEM2);
65     adaptiveStepSimulation("problem2_adaptive.dat", PROBLEM2);
66
67     return 0;
68 }
69
70 void setInitialCondition(
71     const int problem,
72     double pos[NE][N],
73     double vel[NE][N]
74 ) {
75     if (problem == PROBLEM1) {
76         pos[E1][X] = 2.0;
77         pos[E1][Y] = 0.0;
78         pos[E2][X] = -1.0;
79         pos[E2][Y] = 0.0;
80
81         vel[E1][X] = 0.0;
82         vel[E1][Y] = 0.95;
83         vel[E2][X] = 0.0;
84         vel[E2][Y] = -1.0;
85     } else {
86         pos[E1][X] = 1.4;
87         pos[E1][Y] = 0.0;
88         pos[E2][X] = -1.0;
89         pos[E2][Y] = 0.0;
90
91         vel[E1][X] = 0.0;
92         vel[E1][Y] = 0.86;
93         vel[E2][X] = 0.0;

```

```

94         vel[E2][Y] = -1.0;
95     }
96 }
97
98 void calcAcceleration(
99     const double pos[NE][N],
100     double acc[NE][N]
101 ) {
102     const double r1 = sqrt(pos[E1][X] * pos[E1][X] + pos[E1][Y] * pos[E1][Y]
103         );
104     const double r2 = sqrt(pos[E2][X] * pos[E2][X] + pos[E2][Y] * pos[E2][Y]
105         );
106     const double dx = pos[E1][X] - pos[E2][X];
107     const double dy = pos[E1][Y] - pos[E2][Y];
108     const double r12 = sqrt(dx * dx + dy * dy);
109
110     const double r13 = r1 * r1 * r1;
111     const double r23 = r2 * r2 * r2;
112     const double r123 = r12 * r12 * r12;
113
114     acc[E1][X] = -Z * pos[E1][X] / r13 + dx / r123;
115     acc[E1][Y] = -Z * pos[E1][Y] / r13 + dy / r123;
116
117     acc[E2][X] = -Z * pos[E2][X] / r23 - dx / r123;
118     acc[E2][Y] = -Z * pos[E2][Y] / r23 - dy / r123;
119 }
120
121 void eulerRichardsonStep(
122     const double pos[NE][N],
123     const double vel[NE][N],
124     const double dt,
125     double next_pos[NE][N],
126     double next_vel[NE][N]
127 ) {
128     double acc[NE][N];
129     double pos_mid[NE][N];
130     double vel_mid[NE][N];
131     double acc_mid[NE][N];
132
133     calcAcceleration(pos, acc);
134
135     for (int e = 0; e < NE; e++) {
136         for (int k = 0; k < N; k++) {
137             pos_mid[e][k] = pos[e][k] + 0.5 * vel[e][k] * dt;
138             vel_mid[e][k] = vel[e][k] + 0.5 * acc[e][k] * dt;
139         }
140     }
141
142     calcAcceleration(pos_mid, acc_mid);

```

```

141
142     for (int e = 0; e < NE; e++) {
143         for (int k = 0; k < N; k++) {
144             next_vel[e][k] = vel[e][k] + acc_mid[e][k] * dt;
145             next_pos[e][k] = pos[e][k] + vel_mid[e][k] * dt;
146         }
147     }
148 }
149
150 double calcEnergy(
151     const double pos[NE][N],
152     const double vel[NE][N]
153 ) {
154     double kinetic = 0.0;
155
156     for (int e = 0; e < NE; e++) {
157         kinetic += 0.5 * (
158             vel[e][X] * vel[e][X] + vel[e][Y] * vel[e][Y]
159         );
160     }
161
162     const double r1 = sqrt(pos[E1][X] * pos[E1][X] + pos[E1][Y] * pos[E1][Y]
163         );
164     const double r2 = sqrt(pos[E2][X] * pos[E2][X] + pos[E2][Y] * pos[E2][Y]
165         );
166     const double dx = pos[E1][X] - pos[E2][X];
167     const double dy = pos[E1][Y] - pos[E2][Y];
168     const double r12 = sqrt(dx * dx + dy * dy);
169
170     return kinetic - Z / r1 - Z / r2 + 1.0 / r12;
171 }
172
173 double calcDifference(
174     const double pos1[NE][N],
175     const double pos2[NE][N],
176     const double vel1[NE][N],
177     const double vel2[NE][N]
178 ) {
179     double diff = 0.0;
180
181     for (int e = 0; e < NE; e++) {
182         for (int k = 0; k < N; k++) {
183             diff += fabs(pos1[e][k] - pos2[e][k]);
184             diff += fabs(vel1[e][k] - vel2[e][k]);
185         }
186     }
187
188     return diff;
189 }

```

```

188
189 void copyState(
190     const double src_pos[NE][N],
191     const double src_vel[NE][N],
192     double dst_pos[NE][N],
193     double dst_vel[NE][N]
194 ) {
195     for (int e = 0; e < NE; e++) {
196         for (int k = 0; k < N; k++) {
197             dst_pos[e][k] = src_pos[e][k];
198             dst_vel[e][k] = src_vel[e][k];
199         }
200     }
201 }
202
203 void fixedStepSimulation(
204     const char filename[],
205     const int problem
206 ) {
207     FILE *fp = fopen(filename, "w");
208
209     if (fp == NULL) {
210         printf("file open error: %s\n", filename);
211         return;
212     }
213
214     double pos[NE][N];
215     double vel[NE][N];
216     double next_pos[NE][N];
217     double next_vel[NE][N];
218
219     setInitialCondition(problem, pos, vel);
220
221     const double energy0 = calcEnergy(pos, vel);
222     const int max = (int)(T_MAX / DT);
223
224     for (int i = 0; i < max; i++) {
225         const double t = i * DT;
226         const double energy = calcEnergy(pos, vel);
227         const double energy_error = fabs(energy - energy0);
228
229         fprintf(
230             fp,
231             "%f%f%f%f%f%f%f\n",
232             t,
233             pos[E1][X], pos[E1][Y],
234             pos[E2][X], pos[E2][Y],
235             DT,
236             energy,

```

```

237         energy_error
238     );
239
240     eulerRichardsonStep(pos, vel, DT, next_pos, next_vel);
241     copyState(next_pos, next_vel, pos, vel);
242 }
243
244 fclose(fp);
245 }
246
247 void adaptiveStepSimulation(
248     const char filename[],
249     const int problem
250 ) {
251     FILE *fp = fopen(filename, "w");
252
253     if (fp == NULL) {
254         printf("file_open_error: %s\n", filename);
255         return;
256     }
257
258     double pos[NE][N];
259     double vel[NE][N];
260     double full_pos[NE][N];
261     double full_vel[NE][N];
262     double half_pos[NE][N];
263     double half_vel[NE][N];
264     double half2_pos[NE][N];
265     double half2_vel[NE][N];
266
267     setInitialCondition(problem, pos, vel);
268
269     const double energy0 = calcEnergy(pos, vel);
270     double t = 0.0;
271     double dt = DT;
272
273     fprintf(
274         fp,
275         "%f%f%f%f%f%f%f%f\n",
276         t,
277         pos[E1][X], pos[E1][Y],
278         pos[E2][X], pos[E2][Y],
279         dt,
280         energy0,
281         0.0
282     );
283
284     while (t < T_MAX) {
285         if (t + dt > T_MAX) {

```

```

286         dt = T_MAX - t;
287     }
288
289     eulerRichardsonStep(pos, vel, dt, full_pos, full_vel);
290
291     eulerRichardsonStep(pos, vel, 0.5 * dt, half_pos, half_vel);
292     eulerRichardsonStep(half_pos, half_vel, 0.5 * dt, half2_pos,
        half2_vel);
293
294     const double error = calcDifference(full_pos, half2_pos,
        full_vel, half2_vel);
295
296     if (error > ERROR_LIMIT && dt > DT_MIN) {
297         dt *= 0.5;
298         continue;
299     }
300
301     copyState(half2_pos, half2_vel, pos, vel);
302     t += dt;
303
304     const double energy = calcEnergy(pos, vel);
305     const double energy_error = fabs(energy - energy0);
306
307     fprintf(
308         fp,
309         "%f%f%f%f%f%f%f%f\n",
310         t,
311         pos[E1][X], pos[E1][Y],
312         pos[E2][X], pos[E2][Y],
313         dt,
314         energy,
315         energy_error
316     );
317
318     if (error < 0.25 * ERROR_LIMIT && dt < DT_MAX) {
319         dt *= 2.0;
320     }
321 }
322
323 fclose(fp);
324 }

```

5 AI 利用について

本課題では、コードエディタとして Visual Studio Code を使用した。

Visual Studio Code には GitHub Copilot によるコード補完機能が標準で導入されており、コーディング中にサジェスションが表示される場合がある。本課題のプログラムは講義資料の内容に沿って自ら作成した

が、コーディング時にはこれらの補完機能を利用した。

また、演習 3 問題 3 では、適応的時間幅アルゴリズムを用いたシミュレーションを行った。

適応的時間幅アルゴリズムの動作や結果の解釈について理解が十分でなかったため、理解補助を目的として ChatGPT (OpenAI) を利用した。

AI はプログラムの実行や図の生成は行わず、講義内容の理解、実験結果の整理、および結果の解釈の補助として利用した。

実際には長時間にわたり複数回のやり取りを行ったため、以下に、実際に使用したプロンプトと回答を、さらに AI で要約したものを示す。

利用例 1：適応的時間幅アルゴリズムの理解

■プロンプト

演習で適応的時間幅アルゴリズムを使っています。

固定時間幅との違いがよく分かりません。

私にもわかるように説明して下さい。

■AI の回答（要約）

固定時間幅では計算中ずっと同じ時間幅を用いる。

一方、適応時間幅では変化が大きい部分では時間幅を小さくし、変化が小さい部分では時間幅を大きくする。

そのため必要な部分だけ細かく計算できる。

利用例 2：結果図の解釈

■プロンプト

(軌道図を添付)

こんな結果になりました。

これはありますか？

■AI の回答（要約）

軌道図を見る限り、計算自体は正常に行われているように見える。

ただし、適応時間幅アルゴリズムの効果を確認するためには、まず時間幅 dt が実際に変化しているか調べる必要がある。

利用例 3：時間幅の変化について質問

■プロンプト

(dt と時間の関係を表すグラフを添付)
時間幅が変化していることは分かりました。
次は何を確認すればよいですか？

■AI の回答（要約）

時間幅が一定ではなく変化していることは確認できる。
適応時間幅アルゴリズムの効果を評価するためには、固定時間幅の場合と `energy_error` を比較するとよい。

利用例 4：説明が難しかったため追加で質問

■プロンプト

いいですね。
今、私の理解を超えてしまっています。
私にもわかるように説明して下さい。

■AI の回答（要約）

高速道路では周囲の変化が少ないため、大まかに様子を見ても問題ない。
一方、交差点付近では変化が大きいため、より細かく確認する必要がある。
適応時間幅アルゴリズムも同様に、変化が大きい部分では細かく計算し、変化が小さい部分では大きな時間幅で計算していると考えられる。

利用例 5：誤差の比較結果について質問

■プロンプト

(`energy_error` の比較図を添付)
固定時間幅と適応時間幅の `energy_error` を比較したところ、
適応時間幅の方が小さくなりました。
この結果はどのように考察できますか？

■AI の回答（要約）

適応時間幅では、変化の大きい部分で時間幅を小さくして計算するため、誤差が小さくなると考えられる。
今回の結果では、適応時間幅の方が `energy_error` が小さくなっており、固定時間幅よりも精度よく計算できていることが分かる。

利用例 6：考察内容の確認

■プロンプト

Problem1 では最大誤差がかなり小さくなり、
Problem2 でも同様に誤差が小さくなりました。
この結果からどのような考察ができますか？

■AI の回答（要約）

問題 1，問題 2 のどちらについても，適応時間幅を用いた場合は固定時間幅より energy_error が小さくなった。

このことから，適応時間幅アルゴリズムを用いることで，計算誤差を小さくできることが確認できる。

以上のように，AI は講義内容の理解補助および結果の解釈補助のために利用した。